

* Logging
Logging means writing message about what your program is doing.

It helps you:

- Find bugs
- Understand program flow
- Troubleshoot errors in production.

Instead of using Syslog we use Log4j which is powerful and configurable.

Log4j 3 important parts:

- Logger - decides what to log and at what level.
- Appender - decides where to log (console, file, database)
- Layout - decides how the log message looks.

* Log4j Logging Levels (lowest → highest)
DEBUG → INFO → WARN → ERROR → FATAL

i) Debug Level

- Used by developers
- Show detailed internal information
- Mostly use during development

2) Info Level

- Shows normal progress of application.
- Useful for end users & admins.

3) Warn Level

- Something is not ideal, but application still runs
- It's a warning, not a failure.

4) Error Level

- Serious problem
- Application may behave incorrectly.

5) Fatal Level

- Very serious
- Application cannot continue.

Eg. Money Transfer Module.

1) Debug

Helps developer to trace transaction flow
~~logger flow~~
logger.debug(" ");

2) Info

Important for audit & users (Transaction ID)

3) Warn

logger.warn("Transfer delayed due to network failure");

4) Error

logger.error("Transfer failed: insufficient balance");

5) Fatal

logger.fatal("Core banking service unavailable");

* Imp

If logger set to a level, it prints

- That level
- And all higher levels.

Eg if level is Info then

Debug X

Info ✓

Warn ✓

Error ✓

Fatal ✓

* Logger class (org.apache.log4j.*)

logger log = logger.getLogger(MyClass.class)

Now logger is tied to MyClass class.

Appender Interface with different implement

Console Appender → log to console

File Appender → logs to file

JDBC Appender → logs to database

SMTP Appender → Sends to email

Socket Appender → logs to remote server

* Setting root logger

log4j.rootLogger = Warn, stdout, file

* RollingFileAppender

creates a new file based of some condition like file becomes large.

* For pdf

1) com.lowagie.text

a) Document

Represents pdf document

- open() - starts writing to PDF
- add(Element) - adds content (para, table)
- close() - finish and close PDF.

b) PageSize.A4

- sets page size to A4.

c) Paragraph

- Used to add text block

d) Font - Define font style

e) Font Factory

- Factory class to create Font objects.

FontFactory.getFont();

2) com.lowagie.text.pdf

a) PdfWriter.getInstance();

- Connects Document with output stream

b) PdfTable
create table in pdf
addCell()

c) PdfCell
Represents a table cell.

* Jakarta Mail API
jakarta.mail
Session

* Spring Boot Actuator
Spring Boot Actuator is a monitoring and management tool for your application.

What it provides?

Endpoint	Purpose
/actuator/health	Application health
/actuator/info	App. Information
/actuator/beans	All springs beans
/actuator/mappings	All request mappings
/actuator/env	Environment Properties
/actuator/metrics	JVM & app metrics
/actuator/heapdump	JVM heap dump
/actuator/shutdown	stop application

* When a project moves through phases
• Development
• Testing
• Production (Deployment)

It is not require to use same configuration everywhere.

Suppose
Phase

Development
Testing
Production

Database
MySQL
H2
Oracle

If we keep everything in one app-property we would need to edit it again and again which is risky.

Solution → Spring profiles
Spring Boot Provides profile to handle environment specific configuration

* SSO (Single-Sign-On)
login once, access many applications without logging in again

Example

• login with Google
• Then access Gmail, YouTube, Google Drive without re-login

* OAuth2

A security protocol that possible SSO

Client ID

- * Client ID identifies the application publicly while client secret authenticates the app securely with OAuth provider.

* Testing

@Test

is use to mark this as a test case
JUnit will automatically invoke this method

- * JUnitCore is a built-in class in JUnit used to run test classes programmatically. Follow Facade Design Pattern.

To Run only test classes you specify
Execute all @Test methods inside these classes.

Result result = JUnitCore.runClasses(Test.class);

* org.junit.runner	org.junit.runners.Suite
JUnitCore	
Result	
@RunWith	

* @RunWith(Suite.class)

Tells JUnit to use Suite runner

* @Suite.SuiteClasses({Test1.class, Test2.class})

* Mocking

Create a fake (dummy) object instead of using the real object.

why?

- To test class in isolation
- To avoid external dependency
- To make test fast and reliable

Isolation - Testing only one class by itself without using its dependent classes or external systems.

* Mockito -

Java-based testing framework used in unit testing to create mock objects.

Mockito.Mock ();

Example - Method under test

```
public String login(String user, String pass) {
    if (auth.authenticateUser(user, pass)) {
        return "Success";
    } else {
        return "Failure";
    }
}
```

Test with mocha

```
when(authMock.authenticateUser(user, pass)).thenReturn(true);
assertEquals("Success", app.login(user, pass));
```

The Her ~~we~~ if we set theReturn(true) so according to our program if true then it returns (Success) So if it does not return (Success) so ~~or~~ there is bug in our program.